



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Syntax Validator

Curso: Base de Datos II

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Arocutipa Arocutipa, Gian Franco (2023076790)

Soto Oquendo Cristian Gabriel (2026086510)

Tacna – Perú

2026

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	GA, CS	Patrick Cuadros	Patrick Cuadros	04/07/2026	Versión Original

Validador de Sintaxis SQL/NoSQL

Documento de Visión

Versión 1.0

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	GA, CS	Patrick Cuadros	Patrick Cuadros	04/07/2026	Versión Original

INDICE GENERAL

1.	Introducción	1
1.1	Propósito.....	1
1.2	Alcance	1
1.3	Definiciones, Siglas y Abreviaturas.....	1
1.4	Referencias	1
1.5	Visión General	1
2.	Posicionamiento.....	1
2.1	Oportunidad de negocio.....	1
2.2	Definición del problema	2
3.	Descripción de los interesados y usuarios	3
3.1	Resumen de los interesados	3
3.2	Resumen de los usuarios	3
3.3	Entorno de usuario.....	4
3.4	Perfiles de los interesados.....	4
3.5	Perfiles de los Usuarios.....	4
3.6	Necesidades de los interesados y usuarios.....	6
4.	Vista General del Producto	7
4.1	Perspectiva del producto.....	7
4.2	Resumen de capacidades	8
4.3	Suposiciones y dependencias	8
4.4	Costos y precios.....	9

4.5	Licenciamiento e instalación.....	9
5.	Características del producto	9
6.	Restricciones	10
7.	Rangos de calidad.....	10
8.	Precedencia y Prioridad.....	10
9.	Otros requerimientos del producto	10
	b) Estandares legales	32
	c) Estandares de comunicación	37
	d) Estandares de cumplimiento de la plataforma.....	42
	e) Estandares de calidad y seguridad	42
	CONCLUSIONES	46
	RECOMENDACIONES	46
	BIBLIOGRAFIA.....	46
	WEBGRAFIA	46

1. Introducción

1.1 Propósito

El propósito de este documento es recopilar, analizar y definir las necesidades y características de alto nivel del sistema Validador de Sintaxis SQL/NoSQL: una plataforma web, una Skill/API pública y un CLI orientados a la validación sintáctica de consultas SQL multimotor y comandos MongoDB. El documento se centra en la funcionalidad esencial requerida por los interesados y usuarios finales, así como en las razones que motivan su desarrollo, y sirve de referencia para los requerimientos detallados que se especifican en el documento ERS del proyecto.

1.2 Alcance

Este documento de Visión abarca el desarrollo integral del sistema: la plataforma web bajo arquitectura MVC (Express como backend y HTML/JS con Monaco Editor como frontend), la Skill/API pública REST versionada, el CLI instalable sqlcheck, el motor de validación léxico-sintáctico propio para SQL y MongoDB, y el panel administrativo de auditoría. No abarca la ejecución real de las consultas contra un motor de base de datos del usuario, ni la comercialización del producto fuera del ámbito académico del curso.

1.3 Definiciones, Siglas y Abreviaturas

Término	Descripción
SQL	Structured Query Language; lenguaje estándar de consulta para bases de datos relacionales.
NoSQL	Bases de datos no relacionales; en este proyecto representadas por MongoDB.
MVC	Modelo-Vista-Controlador; patrón de arquitectura de software.
AST	Abstract Syntax Tree; estructura usada por un parser para representar la sintaxis de una consulta.
RUP	Rational Unified Process; proceso de desarrollo de software del cual deriva este documento de Visión.
RF / RNF	Requerimiento Funcional / Requerimiento No Funcional.
ERS	Especificación de Requerimientos de Software.
JWT	JSON Web Token; formato de token usado para la autenticación de usuarios.
API	Application Programming Interface; interfaz de comunicación entre sistemas.
CLI	Command Line Interface; interfaz de línea de comandos (sqlcheck).
CI/CD	Integración Continua / Despliegue Continuo; automatización de pruebas y despliegues.
CU	Caso de Uso.

1.4 Referencias

- PostgreSQL Global Development Group. (2024). PostgreSQL Documentation.
- MongoDB Inc. (2024). MongoDB Manual — Query and Projection Operators.
- Microsoft. (2024). Monaco Editor Documentation.
- FD03-EPIS-Informe_SRS_Proyecto-Syntax Validator: Especificación de Requerimientos de Software del sistema Validador de Sintaxis SQL/NoSQL (2026).

1.5 Visión General

Este documento se organiza en nueve secciones. Tras esta introducción, la sección 2 presenta el posicionamiento del producto: la oportunidad y el problema que resuelve. La sección 3 describe a los interesados y usuarios del sistema. La sección 4 ofrece una vista general del producto, su perspectiva, capacidades, suposiciones y modelo de licenciamiento. Las secciones 5 a 9 detallan las características del producto, sus restricciones, los rangos de calidad esperados, la precedencia de las funcionalidades y otros requerimientos. El documento cierra con las conclusiones, recomendaciones y referencias bibliográficas.

2. Posicionamiento

2.1 Oportunidad de negocio

Los equipos de desarrollo que trabajan con múltiples motores de base de datos (MySQL, PostgreSQL, SQL Server, Oracle, SQLite, MongoDB) carecen de una herramienta única que valide la sintaxis de sus consultas antes de ejecutarlas. Existe una fuga constante de tiempo de desarrollo por errores de sintaxis detectados tardíamente y por la dispersión de herramientas de validación por motor. Este sistema representa una ventaja al centralizar la validación multimotor en una sola plataforma accesible desde la web, una API pública y un CLI.

Nº	Objetivo de negocio	Indicador de éxito
1	Ofrecer validación sintáctica multimotor sin necesidad de un motor de base de datos real	Validación exitosa de los 8 motores soportados (SQL ANSI, MySQL, MariaDB, PostgreSQL, SQL Server, Oracle, SQLite, MongoDB)
2	Facilitar la integración externa de la validación mediante una API pública	Adopción de la Skill/API por agentes IA, editores o pipelines de CI/CD
3	Generar historial de validaciones y accesos trazable por usuario	Panel administrativo con auditoría completa de eventos
4	Distribuir la validación como herramienta de línea de comandos	CLI sqlcheck instalable en distribuciones Linux (.deb, .rpm, .tar.gz)

2.2 Definición del problema

El problema de	la falta de una herramienta única que valide la sintaxis de consultas SQL multimotor y MongoDB antes de su ejecución
Afecta a	desarrolladores, administradores de bases de datos y estudiantes que trabajan con distintos motores SQL y MongoDB
El impacto de esto es	ejecución de consultas con errores de sintaxis contra bases de datos reales, pérdida de tiempo en depuración y mensajes de error inconsistentes entre motores

Una solución exitosa debería	permitir validar consultas SQL de distintos dialectos y comandos MongoDB en una sola plataforma, con mensajes de error claros, ubicación exacta y sugerencia de corrección, accesible desde la web, una API pública y un CLI
------------------------------	--

3. Descripción de los interesados y usuarios

3.1 Resumen de los interesados

Nombre	Representa	Rol
Patrick Jose Cuadros Quiroga	Docente del curso Calidad y Pruebas de Software — UPT	Evaluador académico; valida el cumplimiento de los objetivos de calidad y aprueba la documentación del proyecto
Equipo de desarrollo (Gian Franco Arocutipa Arocutipa, Fabrizio Salvador Elias Perez Peralta)	Estudiantes de la Escuela Profesional de Ingeniería de Sistemas	Analizan, diseñan, desarrollan y prueban el Validador de Sintaxis SQL/NoSQL
Administrador de base de datos	Plataforma Railway (PostgreSQL gestionado)	Administra la infraestructura y la base de datos del sistema

3.2 Resumen de los usuarios

Perfil	Descripción	Acceso
Usuario Final	Desarrollador o estudiante que desea validar la sintaxis de una consulta SQL o MongoDB antes de ejecutarla.	Editor web (Monaco), ejemplos de consultas, validación de archivos
Usuario Técnico (Skill/API, CLI)	Agente de IA, editor de código o pipeline de CI/CD que consume la Skill/API pública o el CLI sqlcheck.	Endpoints REST /api/v1, CLI sqlcheck, GitHub Action
Administrador	Usuario con rol admin o superadmin que gestiona usuarios, administradores y consulta la auditoría del sistema.	Panel administrativo web (dashboard, historial, auditoría)

3.3 Entorno de usuario

La plataforma web del Validador de Sintaxis SQL/NoSQL es accesible desde cualquier navegador moderno (Chrome, Firefox, Safari o Edge), con una interfaz responsiva, por lo que no requiere instalación en el equipo del usuario. La Skill/API pública y el CLI sqlcheck, en cambio, se integran directamente en scripts, pipelines o editores de código, sin interfaz gráfica. El usuario final típico interactúa con el editor web de forma esporádica, cada vez que desea validar una consulta, mientras que los consumidores de la Skill/API o el CLI la invocan de forma recurrente y automatizada dentro de sus propios flujos de trabajo.

3.4 Perfiles de los interesados

Docente del curso	
Representante	Patrick Jose Cuadros Quiroga
Descripción	Docente responsable del curso Calidad y Pruebas de Software
Responsabilidades	Definir los criterios de calidad del entregable, revisar avances y aprobar la documentación
Criterios de éxito	El documento de Visión y la ERS reflejan fielmente el alcance y los objetivos del proyecto
Implicación	Revisión periódica de entregables (FD01, FD02, FD03 y subsiguientes)
Restricciones	Disponibilidad limitada a los horarios de clase y asesoría del curso

Equipo de desarrollo	
Representante	Gian Franco Arocutipá Arocutipá, Fabrizio Salvador Elias Perez Peralta
Descripción	Estudiantes de la Escuela Profesional de Ingeniería de Sistemas responsables del análisis, diseño, desarrollo y pruebas del Validador de Sintaxis SQL/NoSQL
Responsabilidades	Construir el sistema cumpliendo los requerimientos funcionales y no funcionales definidos
Criterios de éxito	Entrega de un sistema funcional, desplegado y evaluado positivamente por el docente
Implicación	Desarrollo activo durante todo el semestre académico
Restricciones	Tiempo limitado al cronograma del curso; conocimientos previos en JavaScript, Node.js y bases de datos SQL/NoSQL

3.5 Perfiles de los Usuarios

Usuario Final	
Descripción	Desarrollador o estudiante que desea validar la sintaxis de una consulta SQL o MongoDB sin ejecutarla contra una base de datos real
Tipo	Usuario final
Responsabilidades	Escribir o pegar la consulta en el editor web, seleccionar el motor y revisar los errores reportados
Criterios de éxito	Comprende los errores reportados sin necesitar conocimientos avanzados del motor SQL específico
Implicación	Uso esporádico, orientado a la validación puntual
Restricciones	Conocimientos básicos de SQL o MongoDB

Usuario Técnico (Skill/API, CLI)	
Descripción	Agente de IA, desarrollador de editores o pipeline de CI/CD que integra la Skill/API pública o el CLI sqlcheck en su propio flujo de trabajo
Tipo	Usuario final

Responsabilidades	Invocar los endpoints REST /api/v1 o el CLI sqlcheck; interpretar la respuesta JSON de validación
Criterios de éxito	Obtiene respuestas de validación consistentes y bien documentadas (OpenAPI/Swagger)
Implicación	Uso recurrente y automatizado dentro de pipelines o editores
Restricciones	Requiere conocimientos básicos de consumo de APIs REST

Administrador	
Descripción	Responsable de administrar usuarios, administradores y revisar la auditoría del sistema
Tipo	Usuario técnico
Responsabilidades	Gestionar cuentas de usuario y administrador, consultar estadísticas y el historial de auditoría
Criterios de éxito	El panel administrativo refleja con precisión el estado y la actividad del sistema
Implicación	Intervención periódica para revisión de auditoría y gestión de cuentas
Restricciones	Requiere rol admin o superadmin asignado mediante JWT

3.6 Necesidades de los interesados y usuarios

Necesidad	Prioridad	Inquietudes	Solución actual	Solución propuesta
Validar la sintaxis de una consulta antes de ejecutarla	Alta	Falta de una herramienta única multimotor	Ejecutar la consulta directamente contra el motor real, o ninguna validación previa	Editor web con validación en tiempo real y detección automática de motor
Evitar errores en producción	Alta	Consultas mal formadas (ej. DELETE sin WHERE) ejecutadas por error	Revisión manual o ninguna	Validación previa sin conexión a una base de datos real
Integrar la validación en herramientas externas	Alta	Herramientas dispersas y sin API pública	Reimplementar la gramática SQL en cada integración	Skill/API REST pública versionada y CLI sqlcheck
Dar seguimiento histórico a las validaciones y accesos	Media	Falta de un historial unificado y trazable	Ninguna solución existente	Panel administrativo con auditoría de validaciones, logins y acciones
Aprender sintaxis SQL/NoSQL de forma accesible	Media	Mensajes de error de los DBMS suelen ser crípticos	Documentación dispersa por motor	Mensajes de error con línea, columna y sugerencia de corrección

4. Vista General del Producto

4.1 Perspectiva del producto

El Validador de Sintaxis SQL/NoSQL es un sistema independiente que no requiere integrarse con sistemas externos del usuario. Su arquitectura se organiza bajo el patrón MVC en Node.js: el usuario interactúa con una interfaz web (Vista) que consume una API REST construida en Express (Controlador), la cual delega el análisis de sintaxis a un motor de lexer/parser propio (Modelo, con clases SQLParser, RDPParser y MongoParser) y persiste usuarios, sesiones y auditoría en PostgreSQL. La Skill/API pública y el CLI reutilizan el mismo motor de validación como cliente liviano, enviando el código a analizar al backend y recibiendo un resultado estructurado en JSON.

4.2 Resumen de capacidades

Beneficio para el usuario	Características que lo soportan
Validación sintáctica sin ejecutar la consulta	Lexer y parser propios para SQL multimotor y MongoDB (RF-01 a RF-03)
Detección automática del motor SQL	Sistema de puntuación por palabras clave y tipos de datos (RF-04)
Integración externa sin reimplementar la gramática	Skill/API pública REST versionada y CLI sqlcheck (RF-05, RF-06)
Validación de archivos completos	Procesamiento por streaming con reporte de progreso (RF-07)
Trazabilidad de validaciones y accesos	Panel administrativo con auditoría (RF-08, RF-09)
Autenticación y control de acceso	JWT con roles user/admin/superadmin (RF-10)

4.3 Suposiciones y dependencias

- El equipo de desarrollo cuenta con conocimientos previos de JavaScript, Node.js, Express y bases de datos SQL/NoSQL.
- Las tecnologías utilizadas son de código abierto y no generan costos de licenciamiento.
- Railway ofrece un nivel de facturación por consumo suficiente para el desarrollo y las pruebas académicas del proyecto.
- El motor de validación propio cubre correctamente la sintaxis representativa de los 8 motores soportados durante el desarrollo y las pruebas.
- Se dispone de acceso a Node.js 18+ y a los distintos motores SQL/NoSQL de referencia para validar manualmente el comportamiento del parser durante las pruebas.

4.4 Costos y precios

El Validador de Sintaxis SQL/NoSQL se desarrolla en un entorno académico, sin fines de comercialización inmediata. El stack tecnológico (Node.js, Express, PostgreSQL) es de código abierto y no implica costos de licencia. El despliegue se realiza sobre Railway utilizando facturación por consumo, suficiente para las pruebas y la escala esperada durante el curso.

4.5 Licenciamiento e instalación

La plataforma web no requiere instalación: se accede directamente desde el navegador. La Skill/API pública se consume mediante peticiones HTTP estándar. El CLI sqlcheck se distribuye como paquetes instalables para Linux (.deb, .rpm, .tar.gz), publicados en el repositorio del proyecto. En el contexto académico actual, el sistema se distribuye bajo licencia MIT de código abierto.

5. Característica de Productos

A partir de las necesidades identificadas, el Validador de Sintaxis SQL/NoSQL ofrecerá las siguientes características principales:

- **Autenticación. Registro e inicio de sesión seguro de usuarios y administradores mediante JWT.**
- **Editor web interactivo. Editor Monaco con resaltado de sintaxis, ejemplos de consultas y validación en tiempo real.**
- **Validación SQL multimotor. Detección de errores de sintaxis para SQL ANSI, MySQL, MariaDB, PostgreSQL, SQL Server, Oracle y SQLite, con detección automática del motor.**
- **Validación MongoDB. Análisis de comandos find, insertOne, updateOne y pipelines de aggregate.**
- **Skill/API pública. Endpoints REST versionados (/api/v1) para validar, diagnosticar, corregir, formatear y aplicar lint sobre consultas SQL/NoSQL.**
- **CLI sqlcheck. Validación desde terminal, scripts o pipelines, empaquetado para distribuciones Linux.**
- **Validación de archivos. Procesamiento por streaming de archivos SQL/NoSQL completos, con reporte de progreso.**
- **Panel administrativo. Estadísticas generales, gestión de usuarios y administradores, e historial de auditoría de validaciones y accesos.**

6. Restricciones

- El sistema debe implementarse con arquitectura MVC en Node.js, utilizando Express para el backend.
- La base de datos debe ser PostgreSQL.
- El despliegue debe realizarse sobre Railway, dentro de los límites de su facturación por consumo académico.
- El CLI sqlcheck debe ser compatible con distribuciones Linux (.deb, .rpm, .tar.gz).
- El sistema no debe ejecutar las consultas recibidas contra una base de datos real; solo analiza su sintaxis.
- El desarrollo está limitado al cronograma y los recursos de un equipo de dos estudiantes durante un semestre académico.

7. Rangos de calidad

Categoría	Rango de calidad esperado
Seguridad	Las contraseñas se almacenan cifradas con bcrypt y las sesiones se autentican mediante JWT.
Privacidad	El sistema no persiste el contenido de las consultas validadas; solo se registran metadatos de auditoría (usuario, acción, fecha, IP).
Rendimiento	La validación de una consulta típica se completa en menos de 200 milisegundos.
Disponibilidad	La plataforma web y la Skill/API están disponibles el 99% del tiempo (24/7) en Railway.

Usabilidad	El editor web es responsivo y usable en pantallas de escritorio y portátiles.
Compatibilidad	El validador soporta 8 motores: SQL ANSI, MySQL, MariaDB, PostgreSQL, SQL Server, Oracle, SQLite y MongoDB.
Mantenibilidad	El backend sigue estrictamente el patrón MVC en Node.js (Express) y está cubierto por pruebas unitarias, de mutación y BDD.
Escalabilidad	La arquitectura sin estado (JWT) permite escalar horizontalmente añadiendo más instancias detrás de Railway.
Portabilidad	La plataforma web funciona en Chrome, Firefox, Safari y Edge; el CLI es compatible con distribuciones Linux.

8. Precedencia y Prioridad

La mayoría de los requerimientos funcionales del sistema (autenticación, validación SQL y MongoDB) tienen prioridad Alta, mientras que la Skill/API pública, el CLI y el panel administrativo avanzado tienen prioridad Media. En consecuencia, se propone el siguiente orden de implementación:

Fase	Funcionalidad	Justificación
1	Autenticación y validación SQL/MongoDB (RF-01 a RF-04)	Constituye el núcleo del sistema: sin estas funciones no es posible validar ninguna consulta
2	Panel administrativo y auditoría (RF-08, RF-09)	Permite dar seguimiento y trazabilidad a las validaciones y accesos de la fase 1
3	Skill/API pública y CLI (RF-05, RF-06)	Amplía el alcance del sistema hacia integraciones externas (agentes IA, editores, pipelines)
4	Validación de archivos completos (RF-07)	Funcionalidad de prioridad Media, complementaria para archivos SQL/NoSQL extensos

9. Otros requerimientos del producto

Además de los requerimientos funcionales y no funcionales descritos, el producto debe satisfacer los siguientes estándares transversales:

a) Estándares de manuales de usuario

El editor web debe incluir ejemplos de consultas y mensajes de error en lenguaje claro, de modo que un usuario con conocimientos básicos de SQL o MongoDB pueda comprender y corregir el resultado de su validación sin recurrir a documentación externa.

b) Estándares legales

El sistema debe cumplir con políticas de privacidad de datos: no persistir el contenido de las consultas validadas, informar claramente qué metadatos se registran en la auditoría, y respetar las licencias de código abierto de las librerías utilizadas (Express, jsonwebtoken, bcrypt, todas bajo licencias permisivas tipo MIT).

c) Estándares de comunicación

La comunicación entre el frontend, la Skill/API y el CLI se realiza mediante una API REST con contratos en formato JSON, códigos de estado HTTP estándar y documentación generada mediante OpenAPI/Swagger.

d) Estándares de cumplimiento de la plataforma

La plataforma web debe ser compatible con los navegadores modernos más utilizados (Chrome, Firefox, Safari y Edge) y el CLI debe ser compatible con distribuciones Linux basadas en Debian y Red Hat. El despliegue se realiza sobre la plataforma Railway.

e) Estándares de calidad y seguridad

Las contraseñas deben almacenarse con un esquema seguro (bcrypt), las sesiones deben autenticarse mediante JWT con expiración, y el código debe mantenerse libre de vulnerabilidades conocidas mediante análisis estático (Semgrep) y de dependencias (Snyk) en cada push.

CONCLUSIONES

- El Validador de Sintaxis SQL/NoSQL responde a una necesidad real de los equipos de desarrollo que trabajan con múltiples motores de base de datos, mediante un modelo de validación previa sin necesidad de ejecutar la consulta contra un motor real.
- La arquitectura MVC en Node.js (Express) permite cumplir la visión de un sistema accesible desde cualquier navegador, integrable mediante una API pública y un CLI, y mantenible para el equipo de desarrollo.
- La integración de la Skill/API pública y el CLI amplía la visión del producto más allá del editor web, abarcando también agentes de IA, editores de código y pipelines de CI/CD.
- El uso de tecnologías de código abierto y de la facturación por consumo de Railway hace viable el proyecto dentro de un contexto académico, sin incurrir en costos significativos de licenciamiento.
- La identificación clara de interesados, usuarios y necesidades en este documento de Visión sienta las bases para la Especificación de Requerimientos de Software ya elaborada para el sistema.

RECOMENDACIONES

- Validar periódicamente esta visión con el docente del curso conforme avance el desarrollo, dado que algunos objetivos (nuevas integraciones, nuevos motores SQL) podrían evolucionar.
- Priorizar el desarrollo de las funcionalidades de prioridad Alta antes de abordar las de prioridad Media, conforme al orden propuesto en la sección 8.
- Mantener la coherencia entre este documento de Visión y la Especificación de Requerimientos de Software del proyecto conforme el alcance se vaya refinando.
- Evaluar en iteraciones futuras la incorporación de nuevos motores SQL/NoSQL y la publicación del CLI sqlcheck en gestores de paquetes adicionales (npm, snap), conforme se sugiere en el documento ERS.

BIBLIOGRAFÍA

- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley.
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation.

WEBGRAFÍA

- MongoDB Inc. (2024). MongoDB Manual.
- Node.js Foundation. (2024). Express.js Guide.